PROJECT REPORT

Health360

Smart Healthcare Analytics: Patient Risk & Lifestyle Insights

A web-based healthcare analytics platform built with Python and Flask, featuring interactive Tableau dashboards and data stories to explore patient risk factors, disease patterns, and lifestyle insights.

GitHub Repository: ananyaretarekar/Health360-Flask

TABLE OF CONTENTS

1. Introduction
   1.1 Project Overview
   1.2 Purpose
2. Ideation Phase
   2.1 Problem Statement
   2.2 Empathy Map Canvas
   2.3 Brainstorming
3. Requirement Analysis
   3.1 Customer Journey Map
   3.2 Solution Requirement
   3.3 Data Flow Diagram
   3.4 Technology Stack
4. Project Design
   4.1 Problem Solution Fit
   4.2 Proposed Solution
   4.3 Solution Architecture
5. Project Planning & Scheduling
   5.1 Project Planning
6. Functional and Performance Testing
   6.1 Performance Testing
7. Results
   7.1 Output Screenshots
8. Advantages & Disadvantages
9. Conclusion
10. Future Scope
11. Appendix

1. INTRODUCTION

1.1 Project Overview

Health360 is a web-based healthcare analytics platform developed using Python and Flask that presents interactive Tableau dashboards and data stories on a single, lightweight website. The platform consolidates a 5,000-patient health dataset covering vitals, lifestyle habits, and diagnosed conditions into a connected set of visual dashboards titled "Smart Healthcare Analytics: Patient Risk & Lifestyle Insights."

Rather than requiring end users to open Tableau Desktop or navigate raw spreadsheets, Health360 embeds published Tableau Public views directly inside a Flask-rendered webpage. Visitors can browse age-group demographics, exercise and dietary habits, BMI categories, blood pressure trends, and the prevalence of diabetes, heart disease, and stroke, all within one scrollable, story-driven interface.

1.2 Purpose

The purpose of Health360 is to make population-level health analytics accessible, visual, and easy to interpret for non-technical audiences such as students, healthcare administrators, and the general public. Specifically, the project aims to:

- Translate a raw clinical/lifestyle dataset into an interactive, story-driven analytics experience.
- Highlight correlations between lifestyle factors (smoking, alcohol consumption, exercise level, BMI) and chronic disease prevalence (diabetes, heart disease, stroke).
- Demonstrate how a lightweight Flask application can host and embed Tableau Public dashboards without requiring a heavy backend or database.
- Provide a reusable, deployable template for future health-data dashboards and awareness campaigns.

2. IDEATION PHASE

2.1 Problem Statement

Healthcare and lifestyle data are collected widely, but the insights locked inside them rarely reach the people who could benefit most. Public health datasets are usually shared as static spreadsheets or siloed inside analytics tools that require licences, logins, or technical training to operate. As a result, patients, students, and administrators struggle to answer simple questions such as "Which age group is most at risk for heart disease?" or "How do smoking and alcohol habits change with age?" without specialised tools.

"How might we present patient risk-factor and lifestyle data in a way that is instantly accessible through a web browser, visually engaging, and easy to interpret without prior data-analysis training?"

2.2 Empathy Map Canvas

An empathy map was used to understand the target user — a student, healthcare enthusiast, or administrator exploring population health trends for the first time — before design began.

Quadrant	Observations
Says	"I want to know how my lifestyle habits affect my health risk." "These spreadsheets and raw numbers don't mean anything to me."
Thinks	"There must be a simpler way to see the big picture of patient health trends." "I don't have time to learn a BI tool just to view a few charts."
Does	Searches online for health statistics, screenshots charts from reports, asks a data-savvy colleague to summarise a dashboard.
Feels	Overwhelmed by clinical jargon and dense tables; curious about how their own habits compare to the broader population; reassured by clear visuals.
Pains	Raw datasets are hard to interpret; dashboards are often locked inside Tableau accounts or paid tools; no single page ties the story together.
Gains	A free, browser-based site that opens instantly, tells a clear data story, and lets them explore each dashboard at their own pace.

2.3 Brainstorming

A brainstorming session generated and shortlisted ideas for presenting the dataset effectively:

- Build a single-page dashboard with every chart stacked vertically — rejected as too long and unnavigable.
- Use a tabbed "story" format so related charts (e.g., BMI-related charts, lifestyle charts) are grouped and browsed with next/previous arrows.
- Design each Tableau sheet around one clear question (e.g., "Which BMI category has the highest heart disease rate?") rather than cramming multiple metrics into one view.
- Host the dashboards through a lightweight Flask server so the project needs no database and can be run locally or deployed cheaply.
- Embed Tableau Public views via their official embed/iframe code, with a direct-link fallback in case embedding is blocked.

- Keep a consistent colour theme (amber/gold backgrounds with accent colours per category) across all sheets for visual continuity.

The shortlisted ideas — a Flask-hosted site with a tabbed Tableau story using a consistent visual theme — formed the basis of the proposed solution.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The journey map below traces a first-time visitor's path through Health360, from discovering the link to sharing an insight.

Stage	User Action	Emotion
1. Discover	User hears about Health360 or receives the project link / demo URL.	Curiosity
2. Land	Opens http://127.0.0.1:5000 (or the deployed URL); the Flask homepage loads with the embedded Tableau story.	First impression
3. Explore	Scrolls through the KPI overview, then browses each dashboard tab (Exercise, Blood Pressure, BMI, Alcohol/Smoking, Age Distribution, Fatigue).	Engagement
4. Interact	Hovers over bars/pies for tooltips, uses Tableau's native filters where available, moves between story points with the arrow controls.	Discovery
5. Insight	Identifies patterns, e.g., obese patients show the highest heart-disease rate, or teenagers report the highest alcohol and smoking values.	Understanding
6. Share	Shares the dashboard link or screenshots with peers, or returns later to recheck a statistic.	Satisfaction

3.2 Solution Requirement

Functional Requirements

ID	Requirement
FR-1	Homepage shall load and render within a standard browser without additional plugins.
FR-2	The page shall embed the published Tableau Public story/dashboard via iframe or the Tableau JS embedding API.
FR-3	A direct link to the Tableau Public view shall be provided as a fallback if embedding is blocked.
FR-4	The dashboard shall expose KPIs for Total Patients, Diabetes, Heart Disease, and Stroke counts.
FR-5	Users shall be able to navigate between dashboard sheets using the story's previous/next controls.
FR-6	Charts shall support built-in Tableau interactivity such as tooltips and filters.
FR-7	Static assets (CSS, JS, fonts) shall be served by Flask from the static/ directory.

Non-Functional Requirements

Attribute	Description
Usability	Clean, legible layout with consistent colour coding; no technical setup required to view dashboards.

Attribute	Description
Performance	Homepage and embedded views should load within a few seconds on a standard broadband connection.
Portability	Runs on Windows/macOS/Linux with Python 3 and a virtual environment; minimal dependencies (Flask).
Availability	Depends on Tableau Public's availability for embedded content and on Google Fonts for typography.
Maintainability	Simple Flask structure (app.py, templates/, static/) keeps the codebase easy to extend.
Scalability	Stateless Flask routes mean the app can be containerised or deployed to any WSGI-compatible host.

3.3 Data Flow Diagram

Data flows through the system in four stages: dataset preparation, Tableau workbook authoring, publishing/embedding, and end-user rendering.

- Smart-healthcare CSV dataset (5,000 patient records — demographics, vitals, lifestyle flags, diagnoses)
- ↓ Cleaned and modelled in Tableau Desktop (calculated fields such as BMI Category, Age Group, Health Risk Score)
- ↓ Published as dashboards and a story to Tableau Public
- ↓ Flask app (app.py) serves templates/index.html, which embeds the published Tableau story via the Tableau JS Embedding API
- ↓ Browser renders the Flask page; Tableau's own servers stream the interactive visualization to the client

Because Tableau Public hosts and renders the visuals, Flask's role is limited to serving the static shell page and asset files — no server-side database or API calls are required for the dashboards themselves.

3.4 Technology Stack

Layer	Technology
Backend	Python 3, Flask (routing & template rendering)
Frontend	HTML5, CSS3, JavaScript (templates/index.html, static/css/style.css, static/js/script.js)
Visualization	Tableau Public (dashboards & story, embedded via Tableau JS Embedding API / iframe)
Typography	Google Fonts (loaded over the internet at runtime)
Data Source	CSV-based smart-healthcare dataset (5,000 patient records)
Version Control / Hosting	Git & GitHub (ananyaretarekar/Health360-Flask)
Environment	Python virtual environment (venv); dependencies pinned in requirements.txt

4. PROJECT DESIGN

4.1 Problem – Solution Fit

Aspect	Description
Problem	Health/lifestyle data is hard for non-analysts to interpret from raw tables or locked-away BI tools.
Solution	A free, browser-based site that embeds ready-made Tableau dashboards grouped into a guided story.
Fit	Tableau Public already renders rich, interactive charts for free; Flask only needs to host and embed them, so the fit between the problem (accessibility) and the solution (a simple embedded webpage) requires minimal custom engineering.

4.2 Proposed Solution

Health360 proposes a single Flask web application that acts as a presentation layer for a pre-built Tableau Public story. The story is organised into clearly labelled sheets — Exercise Level Distribution, Average Blood Pressure Across Age Groups, BMI Category vs Heart Disease Rate, Heart Disease by BMI Category and Exercise Level, Heart Disease vs Diabetes Prevalence by BMI Category, Alcohol and Smoking Consumption Across Age Groups, Patient Distribution by Age Group, and Age Group Distribution by Fatigue Condition — plus an overview KPI strip (Total Patients, Diabetes, Heart Disease, Stroke).

The Flask app's only responsibilities are to serve the landing page, load the required static assets (CSS/JS/fonts), and embed the Tableau view with a direct-link fallback for cases where embedding is restricted. This keeps the codebase small (a single app.py route, one template, and a couple of static files) while still delivering a rich, professional analytics experience.

4.3 Solution Architecture

The architecture follows a simple three-tier pattern:

- Presentation Tier: Browser renders templates/index.html, styled by static/css/style.css and scripted by static/js/script.js; Google Fonts are loaded over the internet.
- Application Tier: Flask (app.py) defines the route(s) that render the template and serve static files; no database layer is required.
- Visualization Tier: Tableau Public hosts the published story/dashboards and streams the interactive visuals into the embedded frame, independent of the Flask server.

This separation means the Flask layer can be redeployed or restyled without touching the analytics layer, and the Tableau dashboards can be updated independently by republishing the workbook — the embedded link on the Flask page does not need to change.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was planned and executed in short, sequential sprints, moving from requirement gathering through to documentation and deployment.

Phase	Deliverable	Status
Sprint 1	Requirement gathering, dataset exploration, problem statement & empathy mapping	Completed
Sprint 2	Tableau data modelling — calculated fields (Age Group, BMI Category, Health Risk Score)	Completed
Sprint 3	Dashboard design — building individual sheets (KPI, BMI, exercise, blood pressure, lifestyle)	Completed
Sprint 4	Story assembly — arranging sheets into a guided, tabbed story with navigation	Completed
Sprint 5	Flask application development — app.py route, templates/index.html, embedding logic	Completed
Sprint 6	Styling — static/css/style.css, static/js/script.js, Google Fonts integration	Completed
Sprint 7	Testing (functional & performance), bug fixes, fallback link verification	Completed
Sprint 8	Documentation, GitHub publishing, and final report preparation	Completed

6. FUNCTIONAL AND PERFORMANCE TESTING

Manual testing was carried out across the core user flows to confirm that the embedded analytics experience behaves as intended.

Functional Test Cases

Test Case	Steps	Expected Result	Result
Homepage load	Navigate to http://127.0.0.1:5000	Page renders with embedded Tableau story visible	Pass
Story navigation	Click the ‹ / › arrows on the story toolbar	View switches between dashboard sheets (e.g., Exercise Level → Blood Pressure)	Pass
Chart interactivity	Hover over a bar/pie segment	Tableau tooltip displays underlying values	Pass
Fallback link	Simulate embedding being blocked	Direct Tableau Public link opens the view in a new tab	Pass
Static assets	Inspect network requests for CSS/JS/fonts	style.css, script.js, and Google Fonts load without 404 errors	Pass
Responsiveness	Resize the browser window / view on a smaller screen	Layout and embedded frame scale without breaking	Pass

6.1 Performance Testing

Because the heavy lifting of rendering charts is delegated to Tableau Public, performance testing focused on the Flask shell's responsiveness and the perceived load time of the embedded story.

Metric	Observed Result	Notes
Initial Flask page load	< 1 second	Fast — the shell page has no database queries
Tableau story first render	2–5 seconds	Depends on Tableau Public's own CDN and the visitor's connection
Navigation between story points	< 1 second	Handled client-side by the already-loaded Tableau viz
Concurrent users	Not server-bound	Visualization rendering load is carried by Tableau Public, not the Flask server

7. RESULTS

7.1 Output Screenshots

The following screenshots show the published Tableau dashboards that make up the Health360 story, followed by the combined story view as it renders inside the Flask application.

KPI Overview

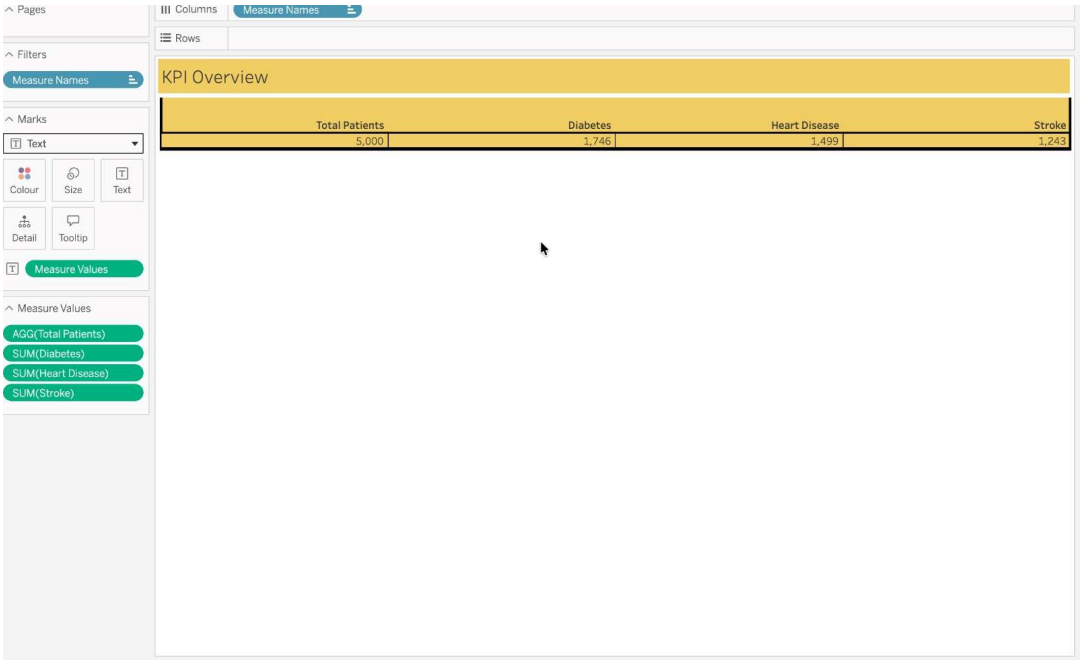


Figure 1: KPI Overview — 5,000 total patients, with 1,746 diabetes, 1,499 heart disease, and 1,243 stroke cases recorded.

Average Blood Pressure Across Age Groups



Figure 2: Average blood pressure is fairly consistent across age groups, ranging from 133.6 (Adult) to 135.7 (Old).

Alcohol and Smoking Consumption Across Age Groups



Figure 3: Teenagers report the highest alcohol (514.0) and smoking (526.0) values of any age group, ahead of Seniors and Adults.

Exercise Level Distribution

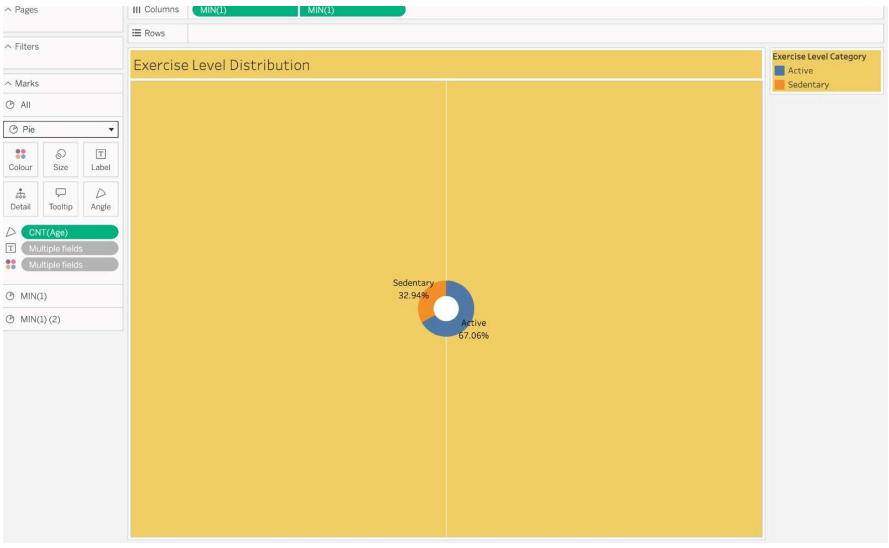


Figure 4: 67.06% of patients are classified as Active, while 32.94% are Sedentary.

Heart Disease vs Diabetes Prevalence by BMI Category



Figure 5: Normal and Overweight BMI categories carry the largest raw counts of diabetes and heart disease cases.

BMI Category vs Heart Disease Rate



Figure 6: Average heart disease rate rises with BMI category — Obese (39.98%) and Overweight (32.91%) exceed Normal (25.97%) and Underweight (20.54%).

Patient Distribution by Age Group

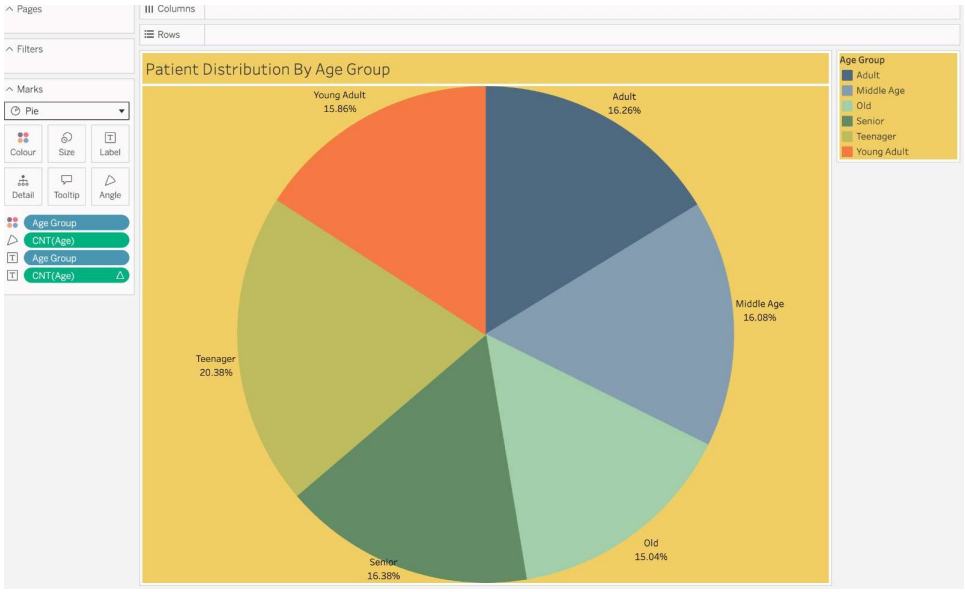


Figure 7: Teenagers form the largest share of patients (20.38%), followed by Senior (16.38%) and Adult (16.26%).

Heart Disease by BMI Category and Exercise Level

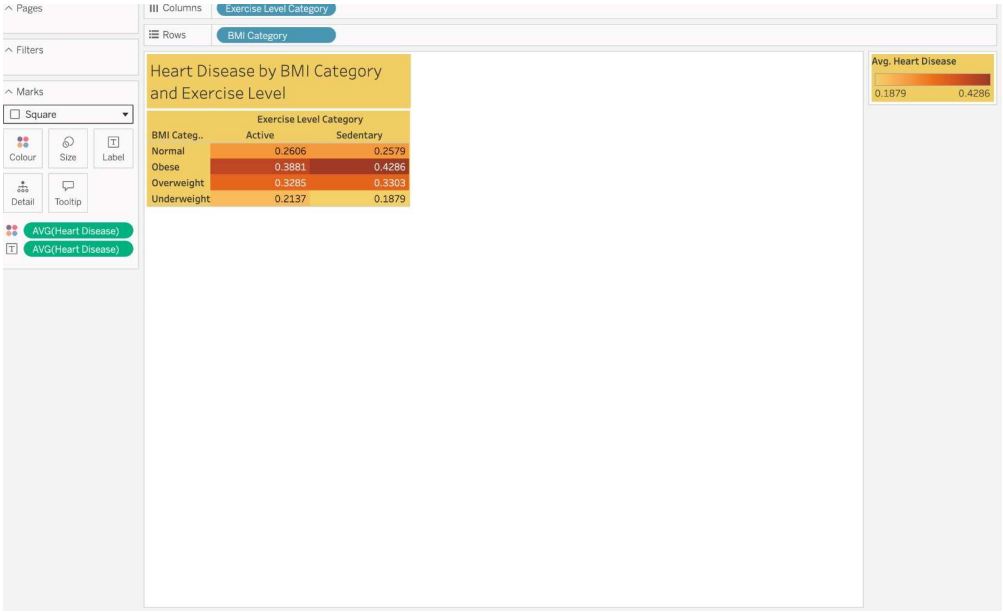


Figure 8: Sedentary, Obese patients show the highest average heart-disease rate (0.4286) of any BMI × exercise combination.

Age Group Distribution by Fatigue Condition



Figure 9: Teenagers report the highest fatigue counts overall, with 521 reporting fatigue versus 498 not reporting it.

Combined Story View (as rendered on the Health360 site)

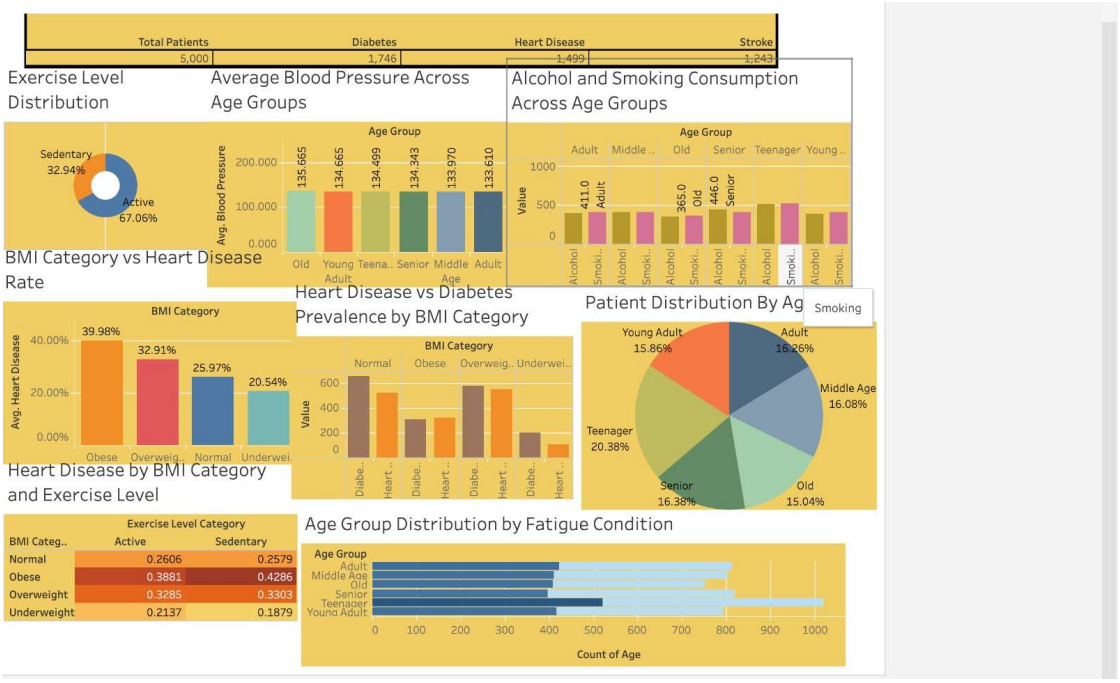


Figure 10: All eight dashboards arranged together as the Health360 analytics story, matching the layout embedded on the Flask homepage.

Story Navigation in the Published Dashboard

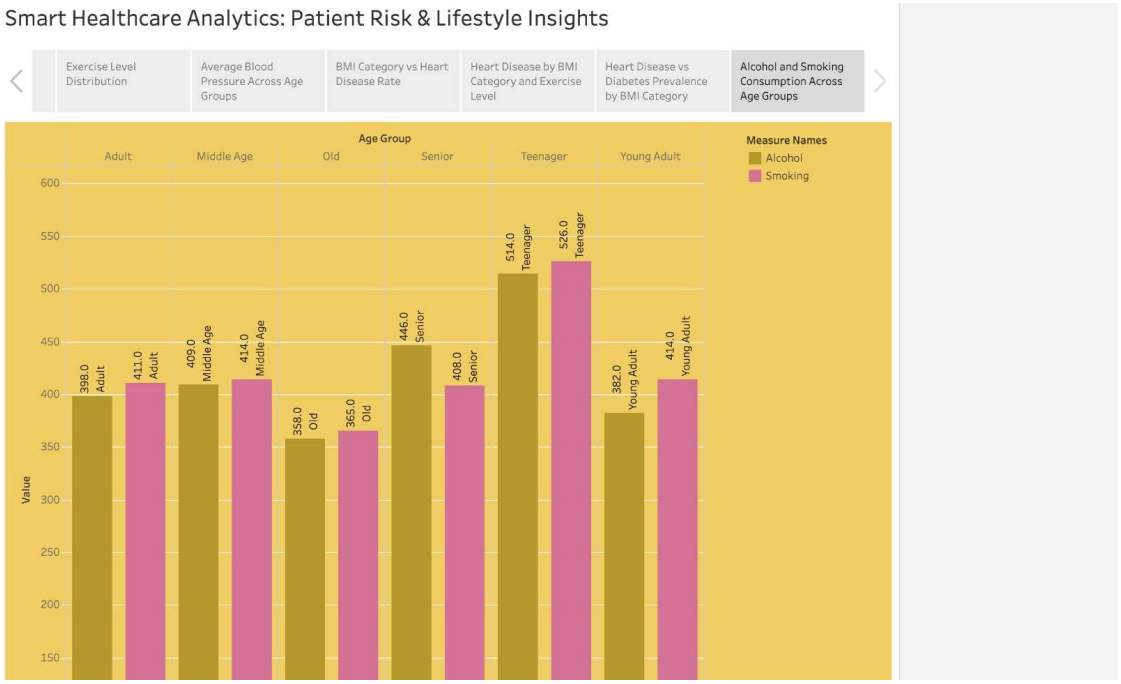


Figure 11: The Tableau story toolbar lets visitors step between sheets — shown here on the "Alcohol and Smoking Consumption Across Age Groups" sheet.

8. ADVANTAGES & DISADVANTAGES

Advantages

- No login, licence, or installation needed — dashboards open directly in a web browser.
- Lightweight Flask backend with no database dependency, making the app easy to run locally or deploy.
- Reuses Tableau Public's native interactivity (tooltips, filtering, story navigation) at no extra development cost.
- A guided "story" format helps non-technical visitors follow a logical narrative instead of a wall of charts.
- Consistent visual theme across dashboards improves readability and professional appearance.
- Clear separation of concerns: Flask serves the shell page while Tableau Public handles all chart rendering.

Disadvantages

- Fully dependent on an active internet connection for both Google Fonts and the embedded Tableau Public content.
- If the workbook owner disables embedding on Tableau Public, the dashboard cannot be shown inline — only the fallback link remains usable.
- Tableau Public dashboards are publicly hosted, so the underlying dataset is visible to anyone who inspects or downloads the workbook.
- Limited customisation of chart behaviour beyond what Tableau's embedding options expose.
- No authentication, personalization, or server-side analytics (e.g., usage tracking) in the current version.

9. CONCLUSION

Health360 demonstrates that a small Flask application, combined with a well-designed Tableau Public story, is enough to turn a raw 5,000-record healthcare dataset into an accessible, browser-based analytics experience. The project successfully surfaces meaningful patterns — such as the link between rising BMI category and heart-disease rate, or the elevated alcohol and smoking figures among teenage patients — without requiring the viewer to touch a spreadsheet or learn a BI tool.

By keeping the backend minimal and delegating visualization rendering to Tableau Public, the project stays easy to maintain, deploy, and extend, while still delivering a polished, interactive front end.

10. FUTURE SCOPE

- Add user authentication and personalised dashboards (e.g., filtering by the visitor's own age group).
- Integrate a live database (PostgreSQL/MySQL) so the dataset can be updated without republishing the Tableau workbook.
- Expand the dataset with additional risk factors (sleep, diet quality, family history) for richer correlation analysis.

- Introduce predictive analytics (e.g., a simple ML risk-score model) alongside the descriptive Tableau dashboards.
- Deploy the Flask app to a cloud platform (Render, Railway, or Azure) with a custom domain for public access.
- Add accessibility improvements (keyboard navigation, screen-reader-friendly summaries of each chart).
- Support multi-language captions for the dashboard narrative to widen the audience.

11. APPENDIX

Source Code

Project structure (Flask application):

- app.py — Flask routes and template rendering
- requirements.txt — Python dependencies
- templates/index.html — homepage template embedding the Tableau story
- static/css/style.css — site styling
- static/js/script.js — client-side behaviour

Dataset Link

Smart Healthcare Analytics dataset — 5,000 patient records covering demographics, vitals, lifestyle habits, and diagnosed conditions (Diabetes, Heart Disease, Stroke); modelled and published via Tableau Public as referenced in the GitHub repository below.

GitHub & Project Demo Link

GitHub Repository: https://github.com/ananyaretarekar/Health360-Flask

Local Demo: Run “python app.py” after installing requirements, then open http://127.0.0.1:5000 in a browser.